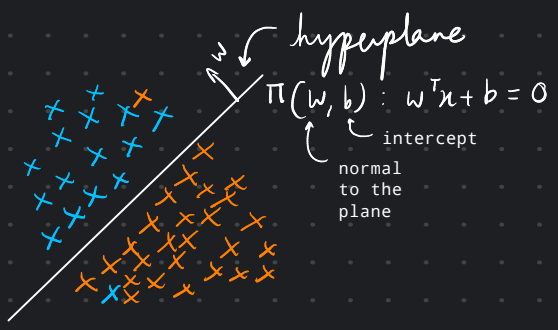
॥ श्री गणेशाय नमः ॥

# Logistic Regression

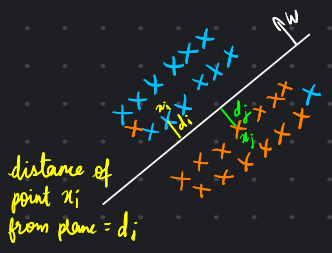



Consider 2 classes of points: -ve (-1) and +ve (1)

If the data is linearly separable, all we have to do is find a hyperplane to separate the classes.

Assumption made when using logistic regression: the classes are almost or fully linearly separable.

Given: $D_n = \{+ve, -ve\}$ To find: $w$, $b$ i.e. the eqn of the hyperplane that best separates the 2 classes.




$$y_i = \begin{cases} +1 & \text{(for +ve points)} \\ -1 & \text{(for -ve points)} \end{cases}$$

$$d_i = \frac{w^T x_i}{\|w\|}$$ (dist. of a pt. from hyperplane)

let's say $w$ is a unit vector. (for simplification) $\|w\| = 1$

$\Rightarrow d_i = w^T x_i > 0$ (since $w$ and $x_i$ are on the same side of the hyperplane)

$\|ly,\ d_j = w^T x_j < 0$ (since $w$ and $x_j$ are on diff. sides of the hyperplane)

$x_i \rightarrow$ +ve point

$x_j \rightarrow$ -ve point

Our classifier would classify a new point based on what side of the hyperplane it lies on.

if $w^T x_i > 0 \longrightarrow y_i = +1$

if $w^T x_i < 0 \longrightarrow y_i = -1$

We make some simplifying assumptions for our hyperplane:
b = 0 (it passes through the origin) and ||w|| = 1 (w is a unit vector).

---

When $y_i = +1$ (+ve point)
and $w^T x_i > 0$ (classified as +ve)
↳ $\boxed{y_i \cdot w^T x_i > 0}$ → correctly classified

When $y_i = -1$ (-ve pt.)
$w^T x_i < 0$ (classified -ve)
↳ $\boxed{y_i \cdot w^T x_i > 0}$ → correctly classified

So $y_i \cdot w^T x_i > 0 \rightarrow$ correct

$y_i \cdot w^T x_i < 0 \rightarrow$ misclassification

When $y_i = +1$ (+ve point)
$w^T x_i < 0$ (classified -ve)
↳ $\boxed{y_i \cdot w^T x_i < 0}$ → incorrectly classified

When $y_i = -1$ (-ve point)
$w^T x_i > 0$ (classified +ve)
↳ $\boxed{y_i \cdot w^T x_i < 0}$ → incorrectly classified

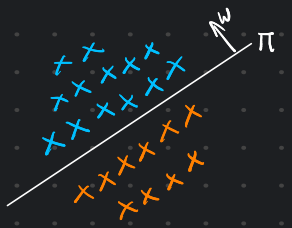
---

As our goal is to maximize correctly classified points, i.e. $\max_w \sum_{i=1}^{n} y_i w^T x_i$. Here $(x_i, y_i)$ is fixed.

$$w^* = \arg\max_w \left( \sum_{i=1}^{n} y_i w^T x_i \right)$$

optimal w

# Squashing and Sigmoid Function




$$w^* = \arg\max \sum_{i=1}^{n} y_i \cdot w^T x_i$$

So there are many hyperplanes $\pi_i$ that could separate the data points, each with a corresponding $w_i$.
Our objective is to find the optimal one.

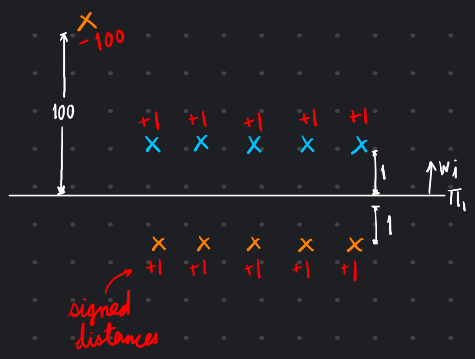
↳ signed distance $\begin{cases} w^T x_i : \text{distance of } x_i \text{ from } \pi \\ y_i : +1 \text{ or } -1 \end{cases}$

$>0 \rightarrow$ correctly classified

$<0 \rightarrow$ misclassified

Our objective would be to maximize the signed distance.

Consider some points:

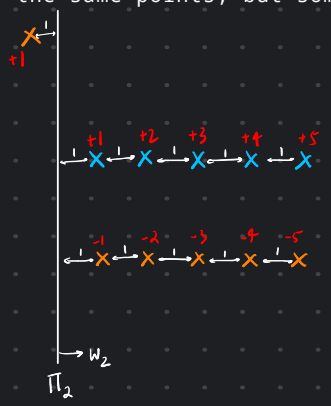



say $\pi_1$ is the separator.

$$\sum_{i=1}^{n} y_i \cdot w^T x_i = 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 - 100 = -90$$

} outlier that is misclassified

Consider the same points, but some other hyperplane:




$$\sum_{i=1}^{n} y_i w_2 x_i = 1 + 2 + 3 + 4 + 5 - 1 - 2 - 3 - 4 - 5 + 1 = 1$$

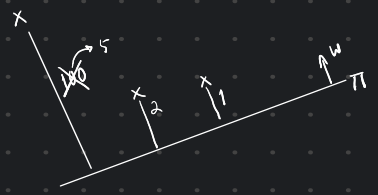
Since our objective is to find a w that maximizes the sum of signed distances, we'd go with the 2nd hyperplane.($\pi_2$)
But

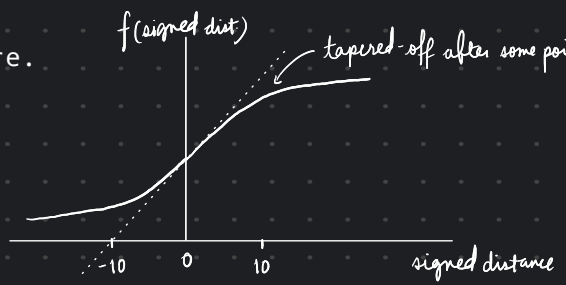
$\text{Accuracy}_{\pi_2} = \frac{6}{11}$ $\quad$ $\text{Accuracy}_{\pi_1} = \frac{10}{11}$ $\quad$ So $\pi_1$ is a better classifier.

One outlier is affecting the model heavily.

## Squashing

If the signed distances are small, we use them as they are.
Else, they are squashed into smaller values.




we'd solve for $\underset{w}{\text{argmax}} \sum_{i=1}^{n} f(y_i w^T x_i)$

## Sigmoid function:

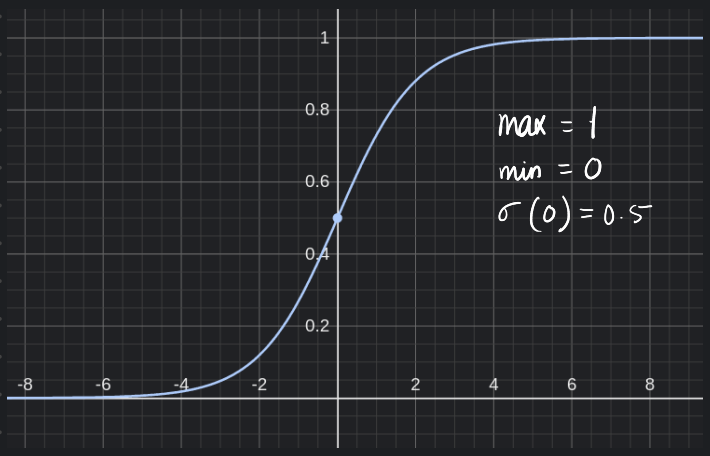
One such function that squashes values is the sigmoid function.

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

A **sigmoid function** is any mathematical function whose graph has a characteristic S-shaped curve or **sigmoid curve**.

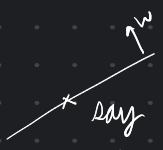
A common example of a sigmoid function is the logistic function shown in the first figure and defined by the formula:[1]

$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x} = 1 - \sigma(-x).$$

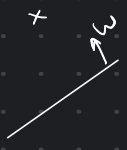



$$w^* = \underset{w}{\text{argmax}} \sum_{i=1}^{n} \sigma(y_i w^T x_i)$$

↳ gives $P(y_i = 1)$




say a point lies on the hyperplane. It could either be +ve/-ve with equal probabilities ($\frac{1}{2}$). (signed dist. = 0)
Also $\sigma(0) = 1/2$




say a point is quite far from the hyperplane and same direction.
$\sigma$(signed dist.) becomes 0.999...

So $\sigma$(signed distance) has a good probabilistic interpretation.

---

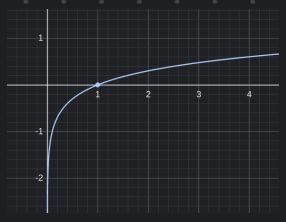
Now we maximize the sum of transformed signed distances.

$$w^* = \underset{w}{\text{argmax}} \sum_{i=1}^{n} \sigma(y_i \cdot w^T x_i) = \underset{w}{\text{arg max}} \sum_{i=1}^{n} \frac{1}{1+\exp(-y_i w^T x_i)}$$

this is less impacted by outliers.

Monotonic functions:
A function g(x) is monotonic if as x increases, g(x) increases as well.

$$\text{if } x_1 > x_2 \longrightarrow g(x_1) \geq g(x_2) \qquad \text{e.g. } \log x$$



Monotonicity of functions could help us simplify optimization problems.

Say our problem is $x^* = \underset{x}{\text{argmin}}\, x^2$



We get min value of $f(x) = x^2$ at $x = 0$

$x^* = 0$

$x^2$ is monotonically increasing when $x > 0$
decreasing when $x < 0$

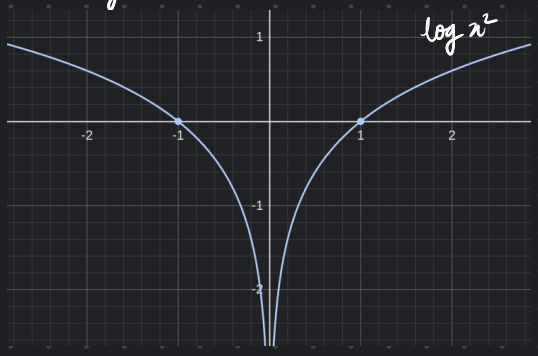
Our problem originally was $x^* = \underset{x}{\text{argmin}}\, f(x)$ ← problem ① where $f(x) = x^2$

let $x' = \underset{x}{\text{argmin}}\, g(f(x))$ ← problem ②

$g(x) = \log x$.

$g(x)$ is monotonically increasing.

$x^* = x'$, as $g(x)$ is monotonic.




If g(x) is a monotonic function,

$$\underset{x}{\text{argmin}}\, f(x) = \underset{x}{\text{argmin}}\, g(f(x))$$

$$\underset{x}{\text{argmax}}\, f(x) = \underset{x}{\text{argmax}}\, g(f(x))$$

---

Mathematical formulation of objective function

$$w^* = \underset{w}{\text{argmax}} \sum_{i=1}^{n} \frac{1}{1+\exp(-y_i w^T x_i)}$$

$$= \underset{w}{\text{argmax}} \sum_{i=1}^{n} \log(\sigma(y_i w^T x_i))$$

$$= \underset{w}{\text{argmax}} \sum_{i=1}^{n} \log\left\{\frac{1}{1+\exp(-y_i w^T x_i)}\right\}$$

$$= \underset{w}{\text{argmax}} \sum_{i=1}^{n} -\log(1+\exp(-y_i w^T x_i))$$

$$\boxed{w^* = \underset{w}{\text{argmin}} \sum_{i=1}^{n} \log(1+\exp(-y_i w^T x_i))}$$

→ similar to $\underset{w}{\text{argmax}} \sum y_i w^T x_i$, except that it doesn't get affected too much by outliers.

NOTE: Here $y_i \in \{-1, 1\}$

An alternate formulation is

$$w^* = \underset{w}{\text{argmin}} \sum_{i=1}^{n} -y_i \log p_i - (1-y_i)\log(1-p_i)$$

$$p_i = \sigma(w^T x_i)$$

here, $y_i \in \{0, 1\}$

Weight vector

$$\downarrow$$

$$w = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log\left(1+\exp(-y_i w^T x_i)\right)$$

$w = \langle w_1, w_2, .., w_d \rangle$ for every feature f1, f2, .., fd, we have a corresponding associated weight.

Say there's a weight $w_i$ corresponding to a feature $x_i$.

case ① If $w_i$ = +ve and $x_{qi} \uparrow$ $\Rightarrow w_i x_{qi} \uparrow$

$\Rightarrow \sum_{i=1}^{d} w_i x_{qi} \uparrow$

$\Rightarrow \sigma(w^T x_q) \uparrow$

$\Rightarrow P(y_q = 1) \uparrow$

case ② If $w_i$ = -ve, $x_{qi} \uparrow$ $\Rightarrow w_i x_{qi} \downarrow$

$\Rightarrow \sum w_i x_{qi} \downarrow$

$\Rightarrow \sigma(w^T x_q) \downarrow$

$\Rightarrow P(y_q = +1) \downarrow$

$\Rightarrow P(y_q = -1) \uparrow$

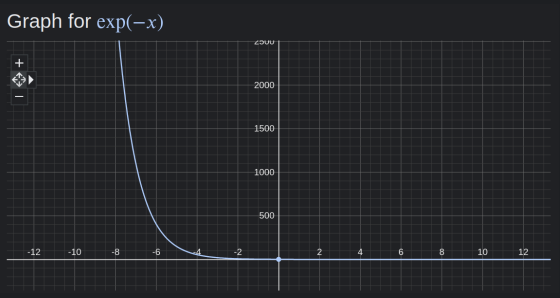
---

## L2 Regularization: Overfitting vs Underfitting

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log\left(1+\exp(-y_i w^T x_i)\right)$$

Let $z_i = y_i w^T x_i$

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log\left(1+\exp(-z_i)\right)$$




Consider $\exp(-z_i)$ $\quad \exp(-z_i) \geq 0$

$\log\left(1 + \underbrace{\exp(-z_i)}_{\geq 0}\right)$ $\quad \geq 1$

log is monotonically increasing. So $\log(1+\delta) \geq \log 1$ if $\delta \geq 0$

So $\log(1+e^{-z_i}) \geq 0$

$\Rightarrow \sum \log(1+\exp(-z_i)) \geq 0$

So the least value it takes is zero. $\quad w^* = \underset{w}{\operatorname{argmin}} \overbrace{\sum \log(1+\exp(-z_i))}^{\downarrow \min = 0}$

As $z_i \to \infty$, $\Rightarrow \exp(-z_i) \to 0$

$\Rightarrow \log(1+\exp(-z_i)) \to 0$

So we get the min. value when $z_i \to \infty \quad \forall i$

$z_i = y_i w^T x_i$ ⟶ here $x_i, y_i$ is given in $D = \{<x_i, y_i>;\ i = 1, .., n\}$

So we need to change $w$ s.t. each $z_i \to \infty^+$

When $z_i$ is +ve $\Rightarrow$ $x_i$ is correctly classified.

i.e. all points are correctly classified.

Problems:
Say we find a w s.t. all the training points are correctly classified. Some of those points could be outliers.
Then we'd have overfitting.
The wi's will tend to either infi or -infi, which are too large or too small.
Solution: regularization

## L2 Regularization

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log(1 + \exp(-y_i w^T x_i)) + \lambda w^T w$$

$$\lambda w^T w = \lambda \sum_{j=1}^{d} w_j^2 = \lambda \|w\|_2^2$$

$\|w\|_2^2$ ← square of $L_2$ norm of $w$.

we're minimizing this as well, so $w$ won't be allowed to be $\pm\infty$ as $\sum w_j^2$ would become $\infty$ as well.

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log(1 + \exp(-y_i w^T x_i)) + \lambda w^T w$$

loss term (logistic loss) | regularization term ($L_2$ regularizer)

here $\lambda$ → hyperparameter.

$\lambda = 0$ → overfit → high variance

$\lambda$ = large → the influence of loss func$^n$ will reduce;
we aren't understanding the training data anymore.
underfitting. → high bias.

we find a good $\lambda$ using cross validation. (same as $k$ in KNN and $\alpha$ in Laplace smoothing)

# L1 regularization and sparsity

$$w^* = \underset{w}{\operatorname{argmin}} \left(\text{logistic loss}\right) + \lambda \|w\|_1 \qquad \|w_i\|_1 = \sum_{i=1}^{d} |w_i|$$

## Sparsity:

Say we have the optimal w = <w1, w2, .., wd>. A solution (w) to LR is said to be sparse if many wi's are 0.

If we use L1 regularization, all the less important features (wi's) become 0.

$$w = \langle w_1, w_2, \ldots, w_d \rangle$$

Say feature fi is unimportant.
If L1 regularization is used, wi becomes 0.
If L2 regularization is used, wi becomes small, but not necessarily 0.

We use L1 regularization if we want sparsity.

---

## Elastic net:

$$w^* = \operatorname{argmin} \sum_{i=0}^{n} \log\left(1 + \exp(-z_i)\right) + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$$

$\lambda_1$ and $\lambda_2$ are hyper parameters.

---

## Probabilistic interpretation

We consider a Gaussian Naive Bayes classifier assuming:
- that if features are real valued, then their probabilities are normally distributed.
- the class labels as Bernoulli rvs (yi = 1 or 0)
- for all i and j, Xi and Xj are conditionally independent given Y (as in NB)

Think of logistic regression as GNB + Bernoulli

$P(x_i|y_i)$ $\quad y_i \sim$ Bernoulli

case : $y_i$ = +ve class

geom: $y_i = +1$

$$w^* = \log\left(1 + \exp(-w^T x_i)\right)$$

prob. : $y_i = 1$

$$w^* = -1 \cdot \log \frac{1}{1+\exp(-w^T x_i)} = \log\left(1 + \exp(-w^T x_i)\right)$$

Both interpretations are same for all cases.

case : $y_i$ = -ve $\qquad$ geom → $y_i = -1$, $\quad$ prob → $y_i = 0$

geom: $\log\left(1 + \exp(w^T x_i)\right)$

prob : $-\log\left(1 - \frac{1}{1+\exp(w^T x_i)}\right)$

$$= -\log \frac{\exp(w^T x_i)}{1+\exp(w^T x_i)}$$

$$= \log \frac{1+\exp(-w^T x_i)}{\exp(-w^T x_i)}$$

$$= \log\left(1 + \frac{1}{\exp(-w^T x_i)}\right) = \log 1 + \exp(+w^T x_i)$$

geom: $w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} \log(1 + \exp(-y_i w^T x_i)) + \text{reg}$ ($y_i$: +1/-1)

prob:

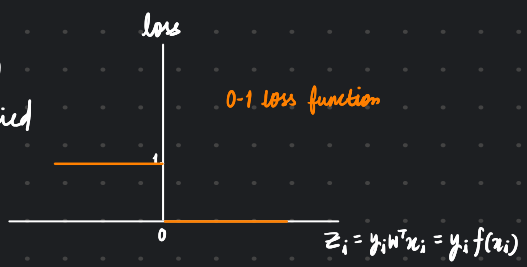
$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} -y_i \log p_i - (1-y_i)\log(1-p_i) + \text{reg}$$

($y_i$: +1/0, $\quad p_i = \sigma(w^T x_i)$)

# Loss function interpretation

Say we give our model +1 points if it incorrectly classifies, and 0 if it correctly classifies a point. (0-1 loss function)
We want to minimize the # of misclassifications.

loss: 0 → correctly classified
1 → misclassified



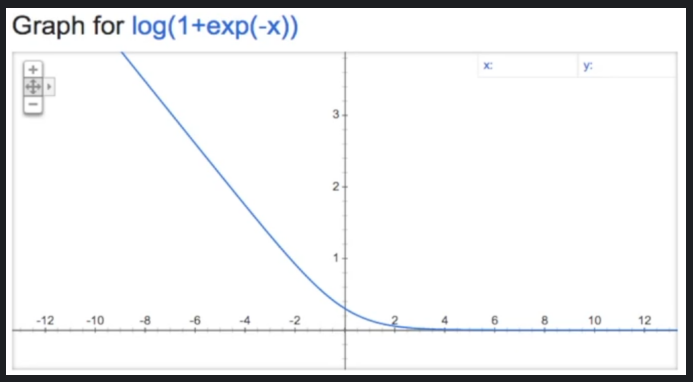

$$0_1(z_i) = \begin{cases} 1, & \text{if } z_i < 0 \\ 0, & \text{if } z_i > 0 \end{cases}$$

An ideal model would be

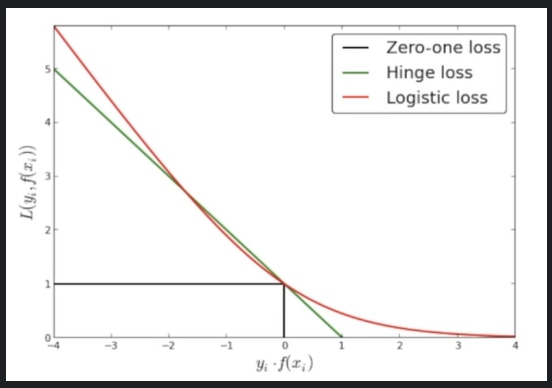
$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} 0_1\,\text{loss}(x_i, y_i, w)$$

we'll need a differentiable loss function to solve the optimization problem, but 0_1 loss is non-continuous (at zi = 0)

We cannot differentiate it, but we could approximate it.
One such approximation is the logistic loss.




logistic loss $= \log\left(\frac{1}{1+e^{-x}}\right)$




loss function

NOTE: Changing the loss function to hinge loss would give SVM, changing it to exponential loss would give AdaBoost, squared loss would give linear regression.

# Hyperparameter Search/ Optimization

We have the hyperparameter $\lambda$.

$\lambda = 0 \rightarrow$ overfitting  $\lambda = 1 \rightarrow$ underfitting

In KNN, since K is an int, we use CV-error for diff. values of K to find the best K.

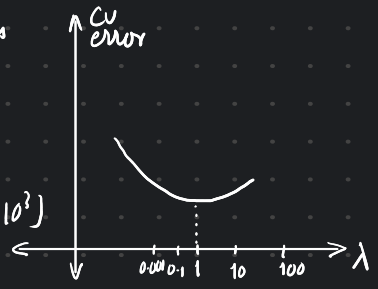
$\lambda$ is a real #. We can't use the same method.

## Grid Search (brute force):

Test with say $\lambda = 0.001$, then $0.01$, then $0.1$, then $1$, then $100$, ..
$[0.001, 0.01, 0.1, 1, 10, 100, 1000]$

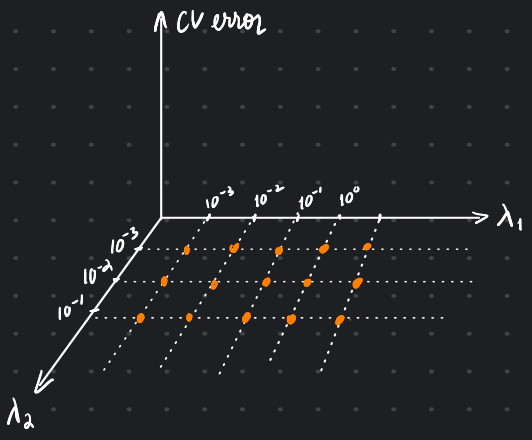
Someone else could test with say $[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

Say we get a plot like this
It tells us that the lowest CV error is at $\lambda \approx 1$




We search in a really wide window
$[10^{-9}, 10^{-3}, 10^{-2}, 10^{-1}, 10^{0}, 10, 10^{2}, 10^{3}]$

If this gives us least CV, the actual best $\lambda$ might be $2.3 \times 10^{-2}$ but that's okay. We're fine with that.




For elastic net, where we have 2 hyperparameters, we check for pairs $(\lambda_1, \lambda_2)$
i.e. all the intersection points on the grid.

Problem:
for $\lambda_1$ → say we check $m_1$ values,
for $\lambda_2$ → say we check $m_2$ values.
We'll have to check for $m_1 * m_2$ values.

So grid search is not good for a large # of hyper parameters.

Say we have 3 hyperparameters, we'd have to train and check $m_1 * m_2 * m_3$ times.

```
# of times the model needs to be trained increases exponentially
as the # of hyperparameters increases.
```

$m^K$ times.

$K$ = # of hyperparameters
$m_1 = m_2 = \ldots = m_K = m$

## Random Search

For one hyperparameter $\lambda \in [10^{-4}, 10^{4}]$, we'll randomly pick points to train & check.

Random search is almost as good as grid search and often faster than grid search.

### sklearn.model_selection.GridSearchCV

*class* `sklearn.model_selection.`**`GridSearchCV`**(*estimator*, *param_grid*, ***, *scoring=None*, *n_jobs=None*, *refit=True*, *cv=None*, *verbose=0*, *pre_dispatch='2*n_jobs'*, *error_score=nan*, *return_train_score=False*) [source]

Exhaustive search over specified parameter values for an estimator.

Important members are fit, predict.

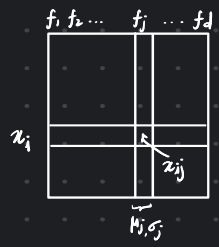
### sklearn.model_selection.RandomizedSearchCV

*class* `sklearn.model_selection.`**`RandomizedSearchCV`**(*estimator*, *param_distributions*, ***, *n_iter=10*, *scoring=None*, *n_jobs=None*, *refit=True*, *cv=None*, *verbose=0*, *pre_dispatch='2*n_jobs'*, *random_state=None*, *error_score=nan*, *return_train_score=False*) [source]

Randomized search on hyper parameters.

RandomizedSearchCV implements a "fit" and a "score" method. It also implements "score_samples", "predict", "predict_proba", "decision_function", "transform" and "inverse_transform" if they are implemented in the estimator used.

# Column/ Feature Standardization




$x_i \in \mathbb{R}^d$

$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$ (standardized $x_{ij}$)

mean centering

scaling

(mean-centering and scaling)

It is reqd. in logistic regression as well since diff. features could have diff. scales.
(we use distances in logistic regression as well).

# Feature importance

We have features f1, f2, ..., fj, ..., fd.

$w_1, w_2, \ldots, w_j, \ldots, w_d$.

[
In KNN, we use forward feature selec^n.
In NB, $P(X_i \mid y=+1)$ → features which are imp.
In LR, $w_j$'s.
]

Assump^n: All features are independent.

↳ Then we can get feature importance by the corresp. weight vectors.

$|w_j|$ = abs. value of wt. corresp. to feature $f_j$.

Case 1: $w_j$ = +ve and large; $\sum_{j=1}^{d} w_j \cdot x_{qj} = w^T x_q \uparrow$

↳ $P(y_q = +1) \uparrow$

Case 2: $w_j$ = -ve and large; $\sum w_j x_{qj} \uparrow$

↳ $P(y_q = -1) \uparrow$

e.g. Say we want to predict "gender" using a # of features. gender = { male (+1), female (-1) }

hl: hair length

$|w_{hl}|$ is large, since hl is an imp. feature to tell the gender.

$w_{hl}$ will be -ve.

↳ As hl ↑, $P(y_q = -1) \uparrow$

Ily, as height ↑ $P(y_q = +1) \uparrow$ ; But height probably isn't as imp as hair length to determine gender.

So $w_{height}$ will be medium & +ve.

# Model interpretability

We can just pick the highest abs weighted features to justify the claim.
E.g. large hair length indicates a greater probability of the person being a female.

So we can give reasoning behind our classification.
The model would be highly interpretable.

# Collinearity or multicollinearity

We'd assumed that the features are independent.
Only then we can use the abs values of weight vectors as feature importance.

Collinearity is one case where the features are not independent.

consider 2 features $f_i, f_j$. If we can get one from the other, by

$f_i = \alpha f_j + \beta$

then $f_i$ & $f_j$ are collinear.

Multicollinearity

$f_1 = \alpha_1 + \alpha_2 f_2 + \alpha_3 f_3 + \alpha_4 f_4 \Rightarrow f_1, f_2, f_3, f_4$ are multicollinear.

Why does $|w_j|$ not indicate feature imp. when there's collinearity?

$D = \langle x_i, y_i \rangle_{i=1}^{n}$ $\quad x_q = \langle x_{q_1}, x_{q_2}, x_{q_3} \rangle$

$w^* = \langle 1, 2, 3 \rangle$ ($f_1$ $f_2$ $f_3$) $\quad w^{*T} x_q = x_{q_1} + 2x_{q_2} + 3x_{q_3}$

If $f_1 = 1.5 f_2 \longrightarrow$ collinear

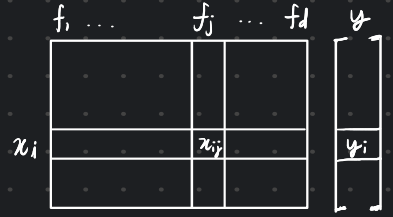
$w^T x_q = x_{q_1} + 3x_{q_1} + 3x_{q_3} = 4x_{q_1} + 3x_{q_3}$

$\langle 4, 0, 3 \rangle$

originally, $w^* = \langle 1, 2, 3 \rangle$, $\tilde{w} = \langle 4, 0, 3 \rangle$ } → same classifier $\therefore f_2 = 1.5 f_1$

So if features are collinear, weight vectors can change arbitrarily.

Perturbation Technique : To determine multicollinearity




Perturbate : to shake a little

for every value $x_{ij}$, add noise $\epsilon \sim N(0, 0.1)$

$x_{ij} + \epsilon$

Before perturbation : $W = \langle w_1, w_2, \ldots, w_j, \ldots, w_d \rangle$

After perturbation : $\tilde{w} = \langle \tilde{w_1}, \ldots, \tilde{w_j}, \ldots, \tilde{w_d} \rangle$

If $w_i$ and $\tilde{w_i}$ differ significantly, then the features are collinear.

If not, you can use them for feature importance.
(abs. values)

If features are collinear, you can still use forward feature selection.

# Time and space complexity

**Training:** solving $w^* = \underset{w}{\text{argmin}}(\text{logistic loss} + \text{reg})$

$T \approx O(nd)$ $\quad n \leftarrow$ # of data pts. $\quad d \leftarrow$ dim. of data

**Runtime:**

At the end of training, we'll get $w^* = \langle w_1, w_2, \ldots, w_d \rangle$

To classify any query pt. $x_q$, we only need to store $w^*$ and do $w^{*T} x_q$

$\text{Space} = O(d)$ ← store $w^* = \langle w_1, \ldots, w_d \rangle$

$w^T x_q = \sum_{j=1}^{d} w_j x_{q_j}$

$\text{Time} = O(d)$

If $d$ is small ⟶ L.R. is really good for low-latency applications.

so it is really quick as well as memory efficient.

---

Say $d$ is large.

$L_1$ regularization creates sparsity. ($w_j$'s corresponding to less imp. features $\to 0$)

$\lambda$ is a reasonable value.

$\lambda \uparrow$ sparsity $\uparrow$

But also, $\lambda \uparrow$ underfitting $\uparrow$

So we have a tradeoff betn bias and latency.

$\lambda \uparrow$ bias $\uparrow$ latency $\downarrow$

Real world cases :

The decision surface for logistic regression is a hyperplane.
Basic assumpⁿ : data is almost linearly separable.

Feature importance and interpretability → $|w_j|$ ⇒ no collinearity

Imbalanced data :
do upsampling

Outliers : less impact due to $\sigma(x)$ but not totally avoided.
$\forall x_i$ calculate $w^{*T} x_i$ (distance) and then remove points that are very far from $\Pi$.

$D_{train} \rightarrow D'_{train}$
$D'_{train} \rightarrow \tilde{w}^*$
↳ final solⁿ

Missing values : Imputation (mean, median, modal)

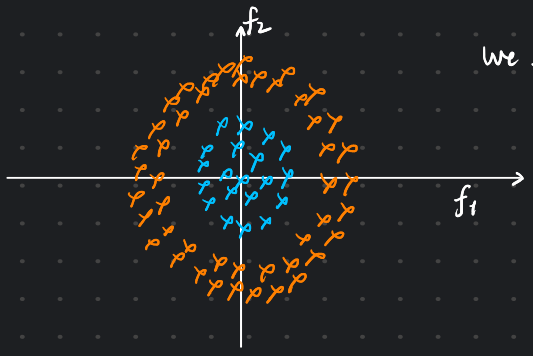
Multiclass extension : one vs rest
maxEnt models ← extensions to LR.
(maximum entropy)
Softmax classifier ⟶ DL
multinomial LR

Similarity matrix : extension → Kernel LR.

Best case : data is linearly separable; low latency requirement;

Large d : chance that data is linearly separable ↑ but latency issues.

# Non-linearly separable data



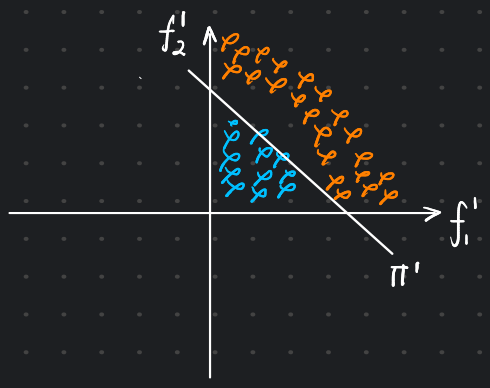

We cannot directly draw a hyperplane to separate classes for data like this.

We can use feature transformation / feature Engineering

$f_1' = f_1^2$ ; $f_2' = f_2^2$

$x_i' = \langle x_{i1}', x_{i2}' \rangle$

$= \langle x_{i1}^2, x_{i2}^2 \rangle$




After feature transform$^n$, we could probably draw a hyperplane to separate the classes.

$f_1, f_2 \rightarrow f_1', f_2'$

we do the same for any given query point as well.

$x_q = \langle x_{q1}, x_{q2} \rangle \xrightarrow{FE/FT} x_q' = \langle x_{q1}', x_{q2}' \rangle = \langle x_{q1}^2, x_{q2}^2 \rangle$

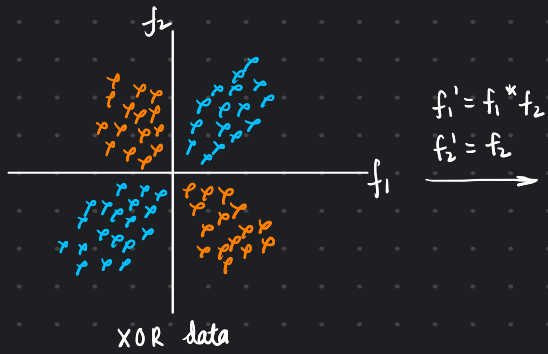
Q. which transformation to apply ?

→ The example data is essentially 2 concentric circles.

eq$^n$ of circle : $f_1^2 + f_2^2 = r^2$ ← not linear in $f_1, f_2$

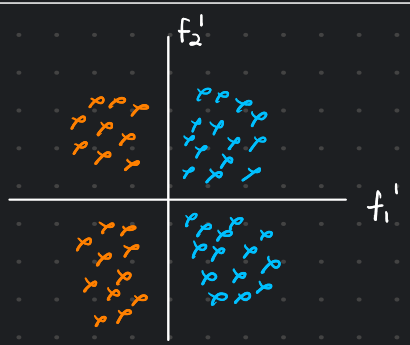
but linear in $f_1^2, f_2^2$.

---

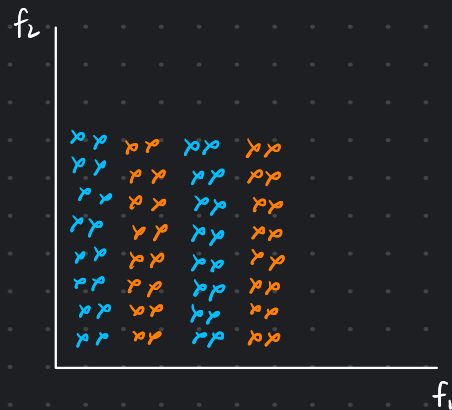



$f_1' = f_1 * f_2$

$f_2' = f_2$

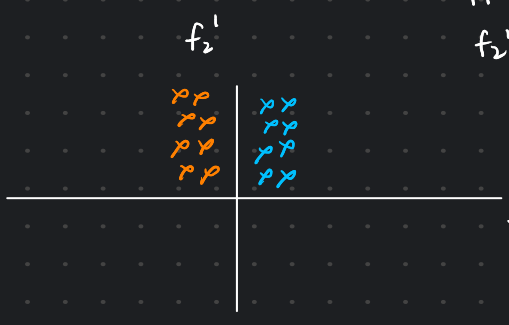



---




$f_1' = \sin f_1$

$f_2' = f_2$



Typical transforms for real-valued features:

① $f_1 * f_2$, $f_1^2$, $f_2^2$, $f_1^3$, ... polynomial features

② Trigonometric features: $\sin(f_1)$, $\cos(f_2)$, $\sin(f_1) * \cos(f_2)$, $\sin^2(f_1)$

③ Boolean features: $f_1$ OR $f_2$, $f_1$ XOR $f_2$.

④ $\log(f_1)$, $e^{f_1}$

---

## Generalized linear models (GLMs)

Logistic regression = GNB + Bernoulli (probabilistic interpretation)

Variations:

Multinomial regression = GNB + Multinomial distributions (softmax) (for multiclass)

Linear regression: $P(Y|X) \sim N(\mu, \sigma^2)$ → regression

Poisson regression: $P(Y|X) \sim Pois(\lambda, k)$

# Code example

```
#code source: http://occam.olin.edu/sites/default/files/DataScienceMaterial
from sklearn.model_selection import train_test_split
from sklearn.grid_search import GridSearchCV
from sklearn.datasets import *
from sklearn.linear_model import LogisticRegression
```

The breast cancer dataset is a classic and very easy binary classification dataset.

| | |
|---|---|
| Classes | 2 |
| Samples per class | 212(M),357(B) |
| Samples total | 569 |
| Dimensionality | 30 |
| Features | real, positive |

```
data = load_breast_cancer() #refer: http://scikit-learn.org/stable/modules/
```

```
tuned_parameters = [{'C': [10**-4, 10**-2, 10**0, 10**2, 10**4]}]
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,
```

## sklearn.linear_model.LogisticRegression

*class* sklearn.linear_model. **LogisticRegression** (*penalty='l2', dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver='liblinear', max_iter=100, multi_class='ovr', verbose=0, warm_start=False, n_jobs=1*) [source]

← $C = \frac{1}{\lambda}$

C ↑ underfitting ↑

← fit intercept

$w^T x + b$

⇒

↑ class weight → resampling

```
#Using GridSearchCV
model = GridSearchCV(LogisticRegression(), tuned_parameters, scoring = 'f1'
model.fit(X_train, y_train)
```

```
print(model.best_estimator_)
print(model.score(X_test, y_test))
```

```
LogisticRegression(C=100, class_weight=None, dual=False, fit_intercept=Tr
ue,
          intercept_scaling=1, max_iter=100, multi_class='ovr', n_jobs=1,
          penalty='l2', random_state=None, solver='liblinear', tol=0.000
1,
          verbose=0, warm_start=False)
0.951219512195
```

```
# More Sparsity (Fewer elements of W* being non-zero) by increasing Lambda

import numpy as np

clf = LogisticRegression(C=0.1, penalty='l1');
clf.fit(X_train, y_train);
w = clf.coef_
print(np.count_nonzero(w))
```

```
8
```

as C ↓ i.e. λ ↑, sparsity ↑ (# of non-zeros ↓)

```
clf = LogisticRegression(C=0.01, penalty='l1');
clf.fit(X_train, y_train);
w = clf.coef_
print(np.count_nonzero(w))
```

```
4
```

```
clf = LogisticRegression(C=0.001, penalty='l1');
clf.fit(X_train, y_train);
w = clf.coef_
print(np.count_nonzero(w))
```

```
3
```

```
clf = LogisticRegression(C=10, penalty='l1');
clf.fit(X_train, y_train);
w = clf.coef_
print(np.count_nonzero(w))
```

```
15
```

λ ↓ sparsity ↑